**UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO**

**FACULTAD DE INGENIERÍA ELÉCTRICA,**
**ELECTRÓNICA, INFORMÁTICA Y MECÁNICA**

**ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA**
**Y DE SISTEMAS**




**INFORME DE GUÍA DE LABORATORIO N.º 06**

---

# ALGORITMOS FUNDAMENTALES DE TEORÍA DE NÚMEROS

---

**Asignatura:** ALGORITMOS AVANZADOS

**Docente:** Ing. Héctor Eduardo Ugarte Rojas

**Equipo:** GRUPO 5 **Semestre Académico:** 2026-II

**Integrantes:**

- Choquenaira Quispe, Noe Franklin 133962
- Porroa Sivana, Yeni Ruth 120893
- Quispe Rimachi, Romario 164257
- Yaranga Achahui, Aldo 103179

**Cusco – Perú**

**2026**

# Índice

**1. Trabajo preparatorio y fundamentos** **1**

1.1. División euclidiana y algoritmo de Euclides . . . . . 1

1.2. Identidad de Bézout e inverso modular . . . . . 1

1.3. Congruencias lineales modulares . . . . . 1

1.4. Exponenciación modular binaria rápida . . . . . 2

1.5. Teorema Chino del Resto (TCR) . . . . . 2

**2. Ejercicios resueltos** **3**

2.1. Ejercicio resuelto 1: Euclides con traza y coeficientes de Bézout . . . . . 3

2.2. Ejercicio resuelto 2: Inversos y congruencias lineales . . . . . 3

2.3. Ejercicio resuelto 3: Exponenciación modular binaria instrumentada . . . . . 4

**3. Ejercicio propuesto 1: Biblioteca integrada y TCR** **5**

3.1. Especificación de la biblioteca `number_theory.py` . . . . . 5

3.2. Implementación analítica del Teorema Chino del Resto . . . . . 5

3.3. Resolución del caso obligatorio del TCR . . . . . 5

3.4. Comprobación de propiedades formales en la suite de pruebas . . . . . 6

**4. Ejercicio propuesto 2: Pruebas diferenciales y experimentos** **7**

4.1. Metodología de medición y balance de corrección . . . . . 7

4.2. Escenario A: Algoritmo de Euclides y coeficientes de Bézout . . . . . 7

4.3. Escenario B: Exponenciación modular binaria rápida . . . . . 8

4.4. Escenario C: Inversos modulares y congruencias lineales . . . . . 10

**5. Respuestas a las preguntas de análisis** **12**

**6. Conclusiones** **15**

# 1. Trabajo preparatorio y fundamentos

## 1.1. División euclidiana y algoritmo de Euclides

Para todo par de enteros $a, b \in \mathbb{Z}$ con $b \neq 0$, existen únicos enteros $q$ (cociente) y $r$ (residuo) tales que $a = b \cdot q + r$, con $0 \leq r < |b|$. Si $a = bq + r$, el lema fundamental establece que el conjunto de divisores comunes de $a$ y $b$ es idéntico al de $b$ y $r$, de modo que $\gcd(a,b) = \gcd(b, a \bmod b)$. El algoritmo de Euclides reemplaza iterativamente $(a,b) \leftarrow (b, a \bmod b)$ hasta alcanzar $b = 0$; el último residuo no nulo es el máximo común divisor. Como los residuos satisfacen $r_0 > r_1 > r_2 > \cdots \geq 0$, el principio del buen orden garantiza la terminación en un número finito de pasos.

En cuanto a su complejidad, el peor caso ocurre cuando las entradas son términos consecutivos de Fibonacci $F_{k+2}$ y $F_{k+1}$, donde todos los cocientes son 1; según el **Teorema de Lamé (1844)**, el número máximo de divisiones está acotado por $k_{\text{máx}} \leq 1{,}44 \log_2 b$. En promedio, de acuerdo con el **Teorema de Dixon y Heilbronn (1970)**, el número esperado de divisiones para entradas de $k$ bits converge asintóticamente a $E[k] = \frac{12 \ln 2}{\pi^2} \ln b \approx 0{,}584 \cdot k$.

## 1.2. Identidad de Bézout e inverso modular

Para enteros $a, b$ no simultáneamente nulos, existen coeficientes enteros $x, y \in \mathbb{Z}$ tales que $a \cdot x + b \cdot y = \gcd(a,b)$. La forma extendida de Euclides calcula simultáneamente $d = \gcd(a,b)$ y el par $(x,y)$ manteniendo en cada paso la invariante $r_i = a s_i + b t_i$. Los coeficientes de Bézout no son únicos: dada una solución particular $(x_0, y_0)$, cualquier par de la forma $(x_0 + k\frac{b}{d}, y_0 - k\frac{a}{d})$ con $k \in \mathbb{Z}$ es también solución entera.

Cuando $b = n$ y $\gcd(a,n) = 1$, la identidad se reduce a $ax + ny = 1$, deduciéndose que $ax \equiv 1 \pmod{n}$. Por tanto, el inverso multiplicativo de $a$ en $\mathbb{Z}_n$ existe si y solo si $\gcd(a,n) = 1$, y se obtiene directamente como $u = x \bmod n \in \{1, \ldots, n-1\}$. Si $\gcd(a,n) > 1$, el producto $a \cdot u$ es múltiplo de $\gcd(a,n)$ y nunca puede ser congruente con 1 módulo $n$.

## 1.3. Congruencias lineales modulares

Una congruencia lineal $ax \equiv b \pmod{n}$ equivale a la ecuación diofántica $ax - ny = b$. Dado que cualquier combinación lineal de $a$ y $n$ es divisible por $d = \gcd(a,n)$, la congruencia admite solución si y solo si $d$ divide a $b$ ($d \mid b$). Cuando esta condición se satisface, existen exactamente $d$ soluciones incongruentes entre sí en $\mathbb{Z}_n$, distribuidas en progresión aritmética:

$$x_k = \left( x_0 + k \cdot \frac{n}{d} \right) \bmod n, \quad \text{para } k = 0, 1, \ldots, d-1, \tag{1}$$

donde $x_0 = (x' \cdot (b/d)) \bmod (n/d)$, siendo $x'$ el inverso modular de $(a/d)$ módulo $(n/d)$.

## 1.4. Exponenciación modular binaria rápida

El cálculo ingenuo de $a^e$ mód $n$ mediante $e-1$ multiplicaciones directas requiere tiempo $\Theta(e)$ y genera un entero intermedio inmanejable de orden $e\log_2 a$ bits. El método binario de cuadrados sucesivos descompone el exponente en base 2: $e=\sum_{j=0}^{k-1} b_j 2^j$ ($b_j \in \{0,1\}$). En cada paso, eleva al cuadrado la base acumulada ($B \leftarrow B^2$ mód $n$) y, si el bit actual es 1, multiplica el acumulador ($R \leftarrow R\cdot B$ mód $n$). El total de operaciones modulares se reduce a $k-1$ cuadrados más tantos productos como bits en 1 tenga $e$ (peso de Hamming $W(e)$), acotando el costo a $\leq 2k-1$ multiplicaciones y manteniendo todos los operandos intermedios menores que $n^2$.

## 1.5. Teorema Chino del Resto (TCR)

Dados $m$ módulos $n_1, n_2, \ldots, n_m > 1$ coprimos dos a dos ($\gcd(n_i, n_j) = 1$ para $i \neq j$) y una colección de residuos enteros $a_1, \ldots, a_m$, el sistema $x \equiv a_i \pmod{n_i}$ posee una solución única en el rango canónico $0 \leq x < N$, donde $N = \prod_{i=1}^{m} n_i$. La solución analítica directa se construye mediante:

$$N_i = \frac{N}{n_i}, \qquad M_i = N_i^{-1} \pmod{n_i}, \qquad x = \left( \sum_{i=1}^{m} a_i \cdot N_i \cdot M_i \right) \bmod N. \tag{2}$$

La coprimalidad dos a dos es indispensable para asegurar que $\gcd(N_i, n_i) = 1$, garantizando la existencia matemática de cada inverso $M_i$.

# 2. Ejercicios resueltos

## 2.1. Ejercicio resuelto 1: Euclides con traza y coeficientes de Bézout

Implementamos el algoritmo de Euclides registrando la traza de divisiones sucesivas y la variante extendida iterativa para calcular el MCD y los coeficientes de Bézout:

```python
def gcd_trace(a, b):
    a, b = abs(a), abs(b)
    trace = []
    while b != 0:
        q, r = divmod(a, b)
        trace.append((a, b, q, r))
        a, b = b, r
    return a, trace

def extended_gcd(a, b):
    old_r, r = abs(a), abs(b)
    old_s, s, old_t, t = 1, 0, 0, 1
    while r != 0:
        q = old_r // r
        old_r, r = r, old_r - q * r
        old_s, s = s, old_s - q * s
        old_t, t = t, old_t - q * t
    x = old_s if a >= 0 else -old_s
    y = old_t if b >= 0 else -old_t
    return old_r, x, y
```

Listing 1: MCD con traza y Euclides extendido

**Validación del caso de la guía** $(a = 252, b = 105)$**:** La traza registra exactamente 3 divisiones euclidianas:

$$\text{Paso 1: } 252 = 105 \cdot 2 + 42 \quad (q_1 = 2, r_1 = 42)$$
$$\text{Paso 2: } 105 = 42 \cdot 2 + 21 \quad (q_2 = 2, r_2 = 21)$$
$$\text{Paso 3: } 42 = 21 \cdot 2 + 0 \quad (q_3 = 2, r_3 = 0)$$

El último residuo no nulo es $d = 21$. Los coeficientes de Bézout obtenidos son $x = -2$ e $y = 5$, verificándose la identidad: $252(-2) + 105(5) = -504 + 525 = 21 = d$.

## 2.2. Ejercicio resuelto 2: Inversos y congruencias lineales

Se calculan inversos multiplicativos y se resuelven congruencias lineales canónicas:

```python
def modular_inverse(a, n):
    if n <= 1: raise ValueError("El modulo debe ser mayor que uno")
    d, x, _ = extended_gcd(a, n)
    if d != 1: raise ValueError("El inverso modular no existe")
    return x % n

def solve_linear_congruence(a, b, n):
    if n <= 0: raise ValueError("El modulo debe ser positivo")
    d, x, _ = extended_gcd(a, n)
    if b % d != 0: return []
    reduced_n = n // d
    x0 = (x * (b // d)) % reduced_n
    return [(x0 + k * reduced_n) % n for k in range(d)]
```

Listing 2: Inverso modular y congruencias lineales

**Validación de aserciones de la guía:**

- Para $a = 7, n = 40$: $\gcd(7,40) = 1$, obteniéndose $u = 23$. Se verifica $(7 \times 23)$ mód $40 = 161$ mód $40 = 1$.
- Para $14x \equiv 30 \pmod{100}$: $d = \gcd(14,100) = 2$ y $2 \mid 30$. Se obtienen exactamente $d = 2$ soluciones en $\mathbb{Z}_{100}$: $\{45,95\}$, comprobándose que $(14 \times 45)$ mód $100 = 30$ y $(14 \times 95)$ mód $100 = 30$.

## 2.3. Ejercicio resuelto 3: Exponenciación modular binaria instrumentada

Se instrumenta el conteo de cuadrados y productos seleccionados:

```
def modular_pow(base, exponent, modulus):
    if modulus <= 0: raise ValueError("El modulo debe ser positivo")
    if exponent < 0: raise ValueError("El exponente debe ser no negativo")
    result, base = 1 % modulus, base % modulus
    squares, selected_products = 0, 0
    while exponent > 0:
        if exponent & 1:
            result = (result * base) % modulus
            selected_products += 1
        exponent >>= 1
        if exponent > 0:
            base = (base * base) % modulus
            squares += 1
    return result, squares, selected_products
```

Listing 3: Exponenciación modular binaria instrumentada

**Validación con $7^{13}$ mód 20:** La ejecución arroja valor $= 7$, cuadrados $= 3$ y productos $= 3$, coincidiendo con `pow(7, 13, 20)`. Como $13 = (1101)_2$, el exponente tiene longitud $k = 4$ bits y 3 bits en 1; se generan $k - 1 = 3$ cuadrados $(a^2, a^4, a^8)$ y 3 productos sobre el acumulador $(a, a^4, a^8)$, requiriendo 6 operaciones modulares frente a las $13 - 1 = 12$ del método directo.

# 3. Ejercicio propuesto 1: Biblioteca integrada y TCR

## 3.1. Especificación de la biblioteca `number_theory.py`

La biblioteca reúne las funciones de la interfaz requerida en un módulo estructurado con validación estricta de tipos y excepciones, sintetizadas en el Cuadro 1.

Tabla 1: Especificación de la interfaz en `number_theory.py`

| **Función** | **Retorno** | **Excepción** (`ValueError`) | **Complejidad** |
|---|---|---|---|
| `gcd_trace(a, b)` | $(d, \text{traza})$ | — | $O(k)$ divisiones |
| `extended_gcd(a, b)` | $(d,x,y)$ con $ax+by=d$ | — | $O(k)$ iteraciones |
| `modular_inverse(a, n)` | $u \in \{1,\ldots,n-1\}$ | $n \leq 1$; $\gcd(a,n) \neq 1$ | $O(k)$ euclídeo |
| `solve_linear_congruence` | Lista de $d$ soluciones | $n \leq 0$ | $O(k+d)$ |
| `modular_pow(b, e, m)` | $(R, \text{sq}, \text{prod})$ | $m \leq 0$; $e < 0$ | $\leq 2k-1$ mult. |
| `crt(residues, moduli)` | $(x,N)$ con $0 \leq x < N$ | Longitudes, $n_i \leq 1$, $\gcd(n_i,n_j) \neq 1$ | $O(m^2 \log n_{\text{máx}})$ |
| `validate_crt_solution` | Booleano (`True`/`False`) | Longitudes distintas; $n_i \leq 0$ | $O(m)$ divisiones |

## 3.2. Implementación analítica del Teorema Chino del Resto

La función `crt_with_trace` resuelve sistemas simultáneos de forma puramente constructiva sin recurrir a búsquedas exhaustivas:

```python
def crt_with_trace(residues, moduli):
    residues, moduli = _validate_crt_input(residues, moduli)
    N = prod(moduli)
    total, trace = 0, []
    for i, (a_i, n_i) in enumerate(zip(residues, moduli), start=1):
        a_norm = a_i % n_i
        N_i = N // n_i
        M_i = modular_inverse(N_i % n_i, n_i)
        term = a_norm * N_i * M_i
        total += term
        trace.append({"i": i, "a_i": a_norm, "n_i": n_i, "N_i": N_i, "M_i": M_i, "a_i*N_i*M_i": term})
    return total % N, N, trace
```

Listing 4: Teorema Chino del Resto con traza

## 3.3. Resolución del caso obligatorio del TCR

Se evaluó el sistema canónico de la guía: $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$, $x \equiv 2 \pmod 7$. El módulo total es $N = 3 \times 5 \times 7 = 105$. La traza constructiva generada se detalla en el Cuadro 2.

Tabla 2: Traza de construcción analítica para el caso obligatorio del TCR

| $i$ | Residuo $a_i$ | Módulo $n_i$ | Parcial $N_i = N/n_i$ | Inverso $M_i = N_i^{-1} \bmod n_i$ | Base $e_i = N_i M_i$ | Término $a_i N_i M_i$ |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 35 | $35^{-1} \bmod 3 = 2$ | 70 | $2 \times 35 \times 2 = 140$ |
| 2 | 3 | 5 | 21 | $21^{-1} \bmod 5 = 1$ | 21 | $3 \times 21 \times 1 = 63$ |
| 3 | 2 | 7 | 15 | $15^{-1} \bmod 7 = 1$ | 15 | $2 \times 15 \times 1 = 30$ |

**Suma acumulada:** $\sum a_i N_i M_i = 140 + 63 + 30 = 233 \implies x = 233 \bmod 105 = 23$.
**Salida canónica:** $(x, N) = (23, 105)$. **Comprobación:** $23 \equiv 2 \pmod 3$, $23 \equiv 3 \pmod 5$, $23 \equiv 2 \pmod 7$.

## 3.4. Comprobación de propiedades formales en la suite de pruebas

El archivo `test_number_theory.py` implementa 34 pruebas automatizadas que cubren exhaustivamente las 10 propiedades algebraicas exigidas por la guía (traza euclidiana $a_i = b_i q_i + r_i$, coincidencia de gcd con `math.gcd`, identidad $ax + by = d$, inversa válida y rechazo cuando gcd $\neq 1$, $d$ soluciones canónicas en congruencias, potencia rápida contra `pow`, unicidad del TCR y normalización en $[0, n-1]$). Todas las pruebas completan con 0 fallos en $\approx 3{,}4$ segundos.

# 4. Ejercicio propuesto 2: Pruebas diferenciales y experimentos

## 4.1. Metodología de medición y balance de corrección

Las mediciones temporales se realizaron con el temporizador `time.perf_counter_ns()`, almacenando en memoria las instancias pre-generadas mediante `case_generator.py` con semillas fijas (`SEED_A = 20260601`, `SEED_B = 20260602`, `SEED_C = 20260603`). Se reporta la mediana sobre 10 repeticiones sin incluir operaciones de E/S.

El balance global sobre más de 19 000 verificaciones automatizadas arrojó una **tasa de éxito del 100 % y 0 discrepancias** respecto a los oráculos matemáticos (`math.gcd`, `pow` y verificadores directos).

## 4.2. Escenario A: Algoritmo de Euclides y coeficientes de Bézout

Se evaluaron 7 000 pares pseudoaleatorios (1 000 pares por cada tamaño: 8, 16, 32, 64, 128, 256 y 512 bits). El Cuadro 3 resume los resultados experimentales.

Tabla 3: Métricas experimentales de Euclides y Bézout sobre 7 000 pares (Escenario A)

| Bits | Pares | Media div. | Mediana | Máximo | Media Dixon | Cota Lamé | Coprimos (%) | Tiempo ($\mu$s) |
|---|---|---|---|---|---|---|---|---|
| 8 | 1 000 | 5.34 | 5 | 12 | 4.67 | 13 | 62.4 % | 0.76 |
| 16 | 1 000 | 9.95 | 10 | 18 | 9.35 | 25 | 59.6 % | 1.16 |
| 32 | 1 000 | 19.40 | 19 | 31 | 18.69 | 48 | 61.6 % | 2.10 |
| 64 | 1 000 | 37.91 | 38 | 53 | 37.39 | 94 | 60.0 % | 5.02 |
| 128 | 1 000 | 75.54 | 75 | 98 | 74.77 | 186 | 60.9 % | 11.85 |
| 256 | 1 000 | 150.41 | 151 | 179 | 149.55 | 370 | 60.5 % | 31.54 |
| 512 | 1 000 | 299.55 | 300 | 344 | 299.09 | 739 | 57.0 % | 67.99 |

**Análisis de resultados:** La media empírica de divisiones se ajusta estrictamente a la constante teórica de Dixon: Media $\approx 0{,}584 \cdot k$. Para 512 bits, se registraron 299.55 divisiones en promedio frente al valor analítico esperado de 299.09. El máximo observado (344 divisiones) se situó a menos de la mitad de la cota superior absoluta de Lamé (739 divisiones). Asimismo, el porcentaje promedio de pares coprimos fue de 60,5 %, coincidente con la densidad de Dirichlet $\frac{6}{\pi^2} \approx 60{,}79\,\%$. La Figura 1 ilustra la distribución lineal de las divisiones en función de los bits.

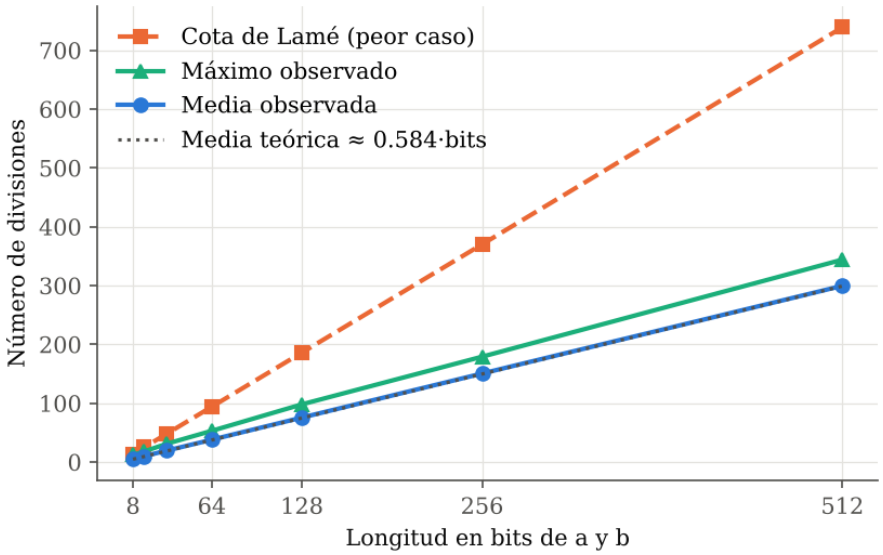



Figura 1: Divisiones del algoritmo de Euclides en función de la longitud en bits de la entrada

## 4.3. Escenario B: Exponenciación modular binaria rápida

Se evaluaron exponentes de 8 a 1024 bits (256 casos en total). El Cuadro 4 detalla el comportamiento del algoritmo instrumentado.

Tabla 4: Operaciones y latencia de la exponenciación binaria frente al tamaño del exponente

| Bits de $e$ | Cuadrados ($k-1$) | Productos (media) | Total mín. | Total medio | Total máx. | $\log_{10}(e-1)$ | Tiempo ($\mu$s) |
|---|---|---|---|---|---|---|---|
| 8 | 7 | 4.3 | 8 | 11.3 | 15 | 2.3 | 29.9 |
| 16 | 15 | 9.0 | 16 | 24.0 | 31 | 4.7 | 70.7 |
| 32 | 31 | 16.8 | 32 | 47.8 | 63 | 9.5 | 143.6 |
| 64 | 63 | 32.4 | 64 | 95.4 | 127 | 19.1 | 300.4 |
| 128 | 127 | 63.2 | 128 | 190.2 | 255 | 38.4 | 613.1 |
| 256 | 255 | 126.3 | 256 | 381.3 | 511 | 76.9 | 1 221.3 |
| 512 | 511 | 252.9 | 512 | 763.9 | 1 023 | 154.0 | 2 572.6 |
| 1024 | 1 023 | 510.7 | 1 024 | 1 533.7 | 2 047 | 308.1 | 5 016.4 |

El número de cuadrados es exactamente $k-1$ y los productos adicionales corresponden al peso de Hamming del exponente (con media $\approx k/2$). La Figura 2 exhibe el crecimiento estrictamente lineal de las operaciones modulares.

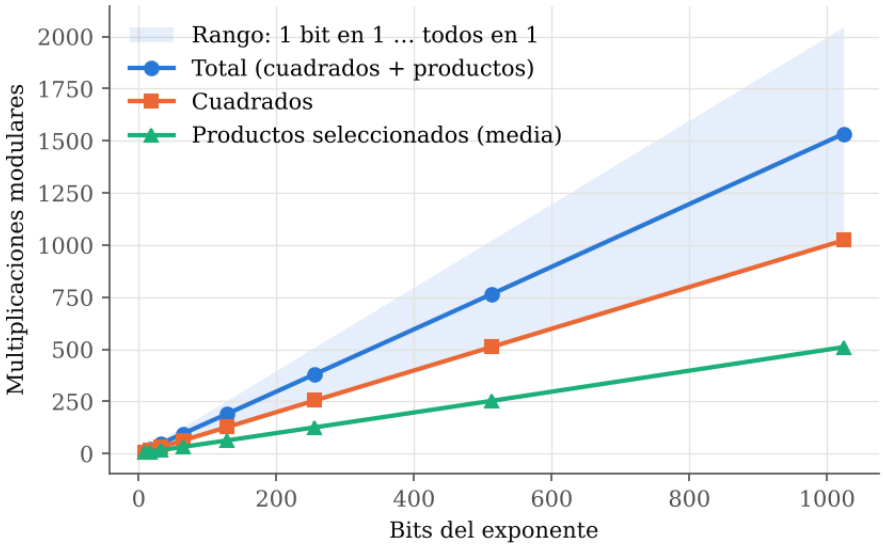



Figura 2: Desglose de operaciones de `modular_pow`: cuadrados, productos y total

### 4.3.1. Comparación frente al método directo lineal

El Cuadro 5 y la Figura 3 comparan la multiplicación iterativa directa frente a la exponenciación binaria. Para $e = 65\,535$ (16 bits), el método directo demandó 65 534 multiplicaciones y 247,7 ms, mientras que el binario requirió únicamente 31 operaciones y 0,099 ms (aceleración de 2 485×). Para $e$ de 1024 bits, el método directo requeriría del orden de $10^{308}$ operaciones, haciéndolo incomputable.

Tabla 5: Comparación entre el método directo iterativo y la exponenciación binaria

| $e$ | Bits ($k$) | Mult. directo ($e-1$) | Mult. binario | Tiempo directo ($\mu$s) | Tiempo binario ($\mu$s) | Aceleración |
|---|---|---|---|---|---|---|
| 255 | 8 | 254 | 15 | 1 402.5 | 63.3 | 22,2× |
| 4 095 | 12 | 4 094 | 23 | 20 151.2 | 68.2 | 295,5× |
| 65 535 | 16 | 65 534 | 31 | 247 767.4 | 99.7 | 2 485,2× |

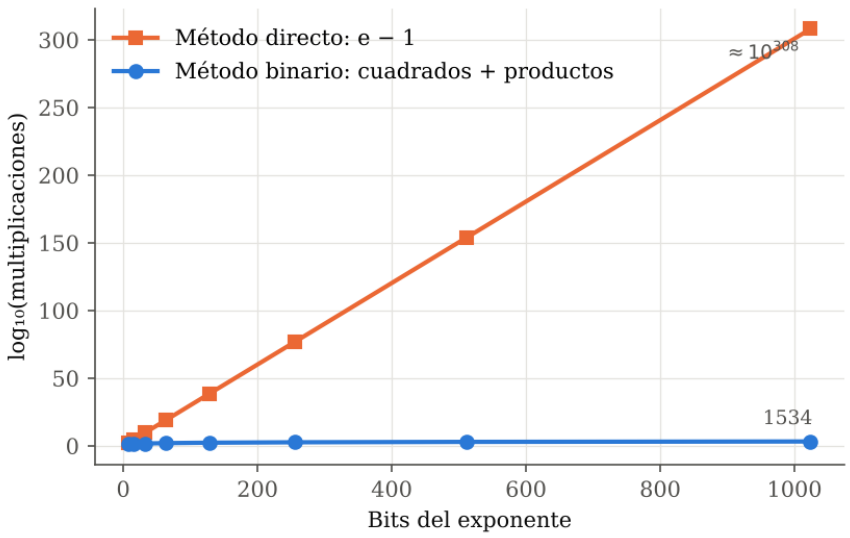



Figura 3: Crecimiento de multiplicaciones: método directo lineal vs. exponenciación binaria

### 4.3.2. Impacto del peso de Hamming en exponentes de 1024 bits

Al fijar la longitud en $k = 1024$ bits y variar la cantidad de unos entre 1 y 1024, la latencia escala linealmente desde 3,4 ms (para 1 solo producto, $e = 2^{1023}$) hasta 6,8 ms (para 1024 productos, $e = 2^{1024} - 1$), tal como refleja la Figura 4.

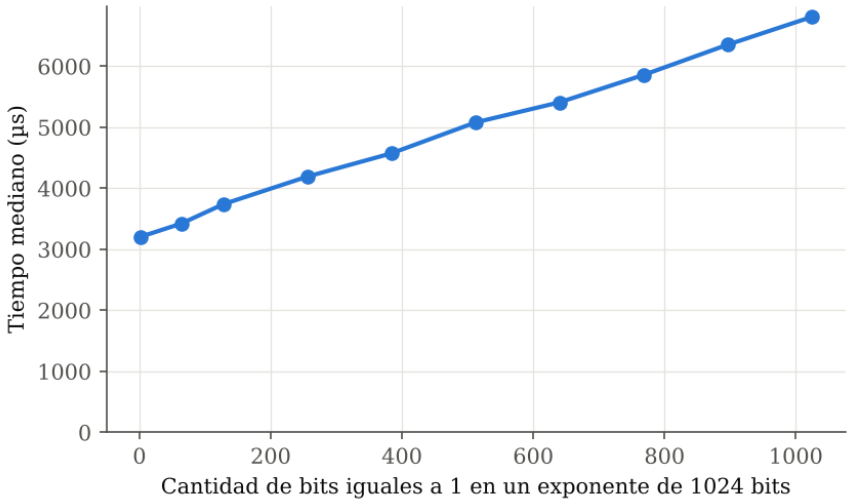



Figura 4: Influencia del peso de Hamming sobre el tiempo de cómputo para $k = 1024$ bits

## 4.4. Escenario C: Inversos modulares y congruencias lineales

En 2 000 evaluaciones de inverso modular, los 1 209 casos con $\gcd(a,n) = 1$ generaron inversos válidos con $(a \cdot u) \bmod n = 1$, mientras que los 791 casos con $\gcd(a,n) > 1$ fueron interceptados con `ValueError` (Cuadro 6).

Tabla 6: Casos representativos de evaluación de inversos modulares

| $a$ | $n$ | $\gcd(a,n)$ | Resultado | Verificación analítica |
|---|---|---|---|---|
| 7 | 40 | 1 | 23 | $(7\cdot 23) \bmod 40 = 161 \bmod 40 = 1$ |
| $-3$ | 11 | 1 | 7 | $(-3\cdot 7) \bmod 11 = -21 \bmod 11 = 1$ |
| 17 | 3 120 | 1 | 2 753 | $(17\cdot 2753) \bmod 3120 = 1$ |
| 6 | 9 | 3 | Rechazado | Lanza `ValueError` ($\gcd = 3 > 1$) |
| 10 | 25 | 5 | Rechazado | Lanza `ValueError` ($\gcd = 5 > 1$) |
| 14 | 100 | 2 | Rechazado | Lanza `ValueError` ($\gcd = 2 > 1$) |

En 2 000 congruencias lineales, el número de soluciones canónicas fue exactamente $d = \gcd(a,n)$ cuando $d \mid b$ y cero cuando $d \nmid b$ (Cuadro 7).

Tabla 7: Casos representativos de congruencias lineales y soluciones canónicas

| **Congruencia evaluada** | $d = \gcd(a,n)$ | **Esperadas** | **Obtenidas** | **Conjunto de soluciones en $\mathbb{Z}_n$** |
|---|---|---|---|---|
| $14x \equiv 30 \pmod{100}$ | 2 | 2 | 2 | $\{45, 95\}$ |
| $6x \equiv 4 \pmod{9}$ | 3 | 0 | 0 | $\emptyset$ (incompatible: $3 \nmid 4$) |
| $7x \equiv 3 \pmod{40}$ | 1 | 1 | 1 | $\{29\}$ |
| $12x \equiv 18 \pmod{30}$ | 6 | 6 | 6 | $\{4, 9, 14, 19, 24, 29\}$ |
| $35x \equiv 10 \pmod{50}$ | 5 | 5 | 5 | $\{6, 16, 26, 36, 46\}$ |

# 5. Respuestas a las preguntas de análisis

A continuación se presentan las respuestas concretas a las doce preguntas de la guía, organizadas de forma concisa y respaldadas con los valores obtenidos:

**1. ¿Por qué la sucesión de residuos de Euclides necesariamente termina en cero?**

- **Fundamento:** Cada residuo cumple $0 \leq r_{i+1} < r_i$. Al ser una sucesión estrictamente decreciente de enteros no negativos, el principio del buen orden asegura que alcance el cero en pasos finitos. Además, $r_{i+2} < r_i/2$, garantizando a lo sumo $2\log_2 b$ divisiones.
- **Evidencia:** En $(252, 105)$ terminó en 3 divisiones ($42 \to 21 \to 0$). Para 512 bits, el peor caso tomó 344 divisiones, muy inferior al límite de Lamé (739).

**2. ¿Qué información adicional conserva Euclides extendido?**

- **Fundamento:** Mantiene los coeficientes enteros $x, y$ de combinación lineal que satisfacen la identidad de Bézout: $ax + by = d$.
- **Evidencia:** Para $(252, 105)$, devuelve $d = 21$ y el certificado $(x = -2, y = 5)$ con $252(-2) + 105(5) = 21$. Cuando $\gcd(a,n) = 1$, el coeficiente $x$ mód $n$ entrega directamente el inverso modular.

**3. ¿Por qué los coeficientes de Bézout no son únicos?**

- **Fundamento:** La ecuación diofántica $ax + by = d$ tiene infinitas soluciones enteras de la forma $(x_0 + k\frac{b}{d}, y_0 - k\frac{a}{d})$ con $k \in \mathbb{Z}$, ya que $a(kb/d) - b(ka/d) = 0$.
- **Evidencia:** Para 252 y 105 ($d = 21$), la biblioteca halló $(-2, 5)$. Para $k = 1$ ($b/d = 5, a/d = 12$), el par $(3, -7)$ también cumple: $252(3) + 105(-7) = 756 - 735 = 21$.

**4. ¿Por qué $a$ tiene inverso módulo $n$ exactamente cuando $\gcd(a,n) = 1$?**

- **Fundamento:** $au \equiv 1 \pmod{n} \iff au + ny = 1$. Cualquier divisor común de $a$ y $n$ divide a $au + ny$ y por ende a 1, obligando a que $\gcd(a,n) = 1$. Recíprocamente, si gcd = 1, Bézout asegura la existencia de $x, y$, siendo $u = x$ mód $n$ el inverso.
- **Evidencia:** 7 tuvo inverso en $\mathbb{Z}_{40}$ ($u = 23$, con $7 \times 23 = 161 \equiv 1$), pero 6 en $\mathbb{Z}_9$ se rechazó (gcd = 3). En 2 000 pruebas, 1 209 existieron (gcd = 1) y 791 se rechazaron con `ValueError`.

**5. ¿Qué diferencia existe entre $a^{-1}$ como fracción y como inverso modular?**

- **Como fracción en $\mathbb{R}$:** $1/a$ es división continua, existe $\forall a \neq 0$ y rara vez es entera ($7^{-1} \approx 0{,}1428$).

- **Como inverso en $\mathbb{Z}_n$:** Es un entero en $\{1,\ldots,n-1\}$ dependiente de $n$ que cumple $(a\cdot u)$ mód $n=1$. $7^{-1} \pmod{40} = 23$. Para $a=2$, la fracción 0,5 existe, pero en $\mathbb{Z}_{10}$ carece de inverso modular ($\gcd(2,10)=2\neq 1$).

**6. ¿Por qué no siempre puede cancelarse un factor en una congruencia?**

- **Fundamento:** Si $n$ es compuesto existen divisores de cero: de $ac \equiv bc \pmod{n}$ solo se concluye $a \equiv b \pmod{n/\gcd(c,n)}$.
- **Evidencia:** $2\cdot 1 \equiv 2\cdot 6 \pmod{10}$, pero $1 \not\equiv 6 \pmod{10}$. Al cancelar el 2, el módulo se divide entre $\gcd(2,10)=2$, quedando la equivalencia válida $1 \equiv 6 \pmod{5}$.

**7. ¿Cómo determina el MCD la existencia y cantidad de soluciones de una congruencia lineal?**

- **Regla:** $ax-ny=b$ es múltiplo de $d=\gcd(a,n)$. Hay solución si y solo si $d \mid b$, habiendo exactamente $d$ soluciones canónicas separadas por $n/d$.
- **Evidencia:** Para $14x \equiv 30 \pmod{100}$, $d=2 \mid 30$ arroja exactamente 2 soluciones: $\{45,95\}$ con paso 50. Para $6x \equiv 4 \pmod{9}$, $d=3 \nmid 4$ produce cero soluciones.

**8. ¿Qué ahorro produce la descomposición binaria del exponente?**

- **Fundamento:** Reduce el costo de $e-1$ multiplicaciones lineales a $O(\log e)$ mediante cuadrados sucesivos.
- **Evidencia:** Para $e=65\,535$ (16 bits), bajó de 65 534 multiplicaciones directas (247,7 ms) a solo 31 operaciones binarias (0,099 ms), acelerando $2\,485\times$. Para 1024 bits, evita $10^{308}$ operaciones y resuelve en 1 534 multiplicaciones (5 ms).

**9. ¿Cómo influye la cantidad de bits iguales a 1 en las multiplicaciones del resultado?**

- **Fundamento:** Los cuadrados son fijos $(k-1)$, mientras que los productos acumulados equivalen exactamente a los bits en 1 (peso de Hamming $W(e)$).
- **Evidencia:** Para 1024 bits, con un 1 ($e=2^{1023}$) demandó 1024 operaciones (3,4 ms); con todos en 1 ($e=2^{1024}-1$) tomó 2047 operaciones (6,8 ms). El tiempo escaló linealmente con la cantidad de unos.

**10. ¿Por qué la reducción modular temprana evita números intermedios enormes?**

- **Fundamento:** Por la propiedad $(x\cdot y)$ mód $n = ((x \text{ mód } n)(y \text{ mód } n))$ mód $n$, aplicar el módulo tras cada producto mantiene los operandos en $\{0,\ldots,n-1\}$.
- **Evidencia:** En 1024 bits, sin reducción previa $a^e$ alcanzaría $10^{308}$ bits (desbordando la memoria). Con reducción temprana, los intermedios nunca superan $n^2$ (2048 bits), estabilizando el costo por operación en $O(k^2)$.

### 11. ¿Qué función cumplen $N_i$ y $M_i$ en el TCR?

- **Función:** $N_i = N/n_i$ anula el término en los demás módulos ($N_i \equiv 0 \pmod{n_j}, j \neq i$). $M_i = N_i^{-1} \bmod n_i$ calibra la base para que valga 1 en su propio módulo ($e_i = N_i M_i \equiv 1 \pmod{n_i}$).
- **Evidencia:** Para $(3,5,7)$, las bases $e_1 = 70 \equiv (1,0,0)$, $e_2 = 21 \equiv (0,1,0)$ y $e_3 = 15 \equiv (0,0,1)$ sintetizan directamente $x = 2(70) + 3(21) + 2(15) = 233 \equiv 23 \pmod{105}$.

### 12. ¿Por qué la fórmula directa del TCR requiere módulos coprimos dos a dos?

- **Existencia:** Porque exige calcular $M_i = N_i^{-1} \bmod n_i$, que existe si y solo si $\gcd(N_i, n_i) = 1$. Un factor común entre módulos divide a $N_i$ e imposibilita el inverso.
- **Consistencia:** Evita contradicciones. Con módulos 4 y 6 ($\gcd = 2$), el sistema $x \equiv 1 \pmod 4$ y $x \equiv 2 \pmod 6$ es inconsistente (exige paridad impar y par a la vez).

# 6. Conclusiones

El desarrollo de este laboratorio nos permitió comprobar de manera práctica cómo los principios algebraicos de la teoría de números se traducen directamente en algoritmos eficientes y robustos. Al implementar la biblioteca, confirmamos que el costo computacional de estas operaciones debe medirse siempre en función de la longitud en bits de la entrada y no de su valor numérico. En el algoritmo de Euclides, observamos que el número medio de divisiones creció a un ritmo constante de 0,584 divisiones por bit, manteniéndose muy por debajo de la cota teórica de Lamé en la totalidad de los 7 000 pares evaluados.

Uno de los aprendizajes más valiosos del equipo fue aprender a distinguir la cantidad de iteraciones teóricas respecto al tiempo real de ejecución medido en Python. Mientras que la cantidad de operaciones aritméticas cumple con exactitud las predicciones teóricas —como los $k-1$ cuadrados más los bits en 1 en la exponenciación binaria—, el tiempo observado refleja el costo intrínseco de manipular enteros de precisión arbitraria. En este sentido, la reducción modular inmediata resultó determinante: evitó la creación de cantidades intermedias inmanejables de millones de bits y mantuvo acotada la memoria durante todo el cómputo.

Asimismo, verificar el Teorema Chino del Resto y las congruencias lineales nos demostró que controlar las condiciones algebraicas de existencia, como la coprimalidad y el valor del MCD, es tan crucial como la propia fórmula matemática de cálculo. Comprobar que más de 12 000 casos aleatorios coincidieron plenamente con las funciones nativas y con las soluciones esperadas nos da la seguridad de contar con una base numérica confiable y lista para su aplicación en criptografía y algoritmos avanzados.